

Recurrent Neural Network (RNN)

Introduction

- Recurrent Neural Networks (RNNs): Understanding Sequences, Memory, and Learning Over Time

Motivation

- How can machines understand *time* — not just static data?
- predicting the next word in a sentence, stock prices
- Imagine you're watching a movie. To understand the current scene, you recall what happened earlier.
 - Let's see how RNNs make this possible.

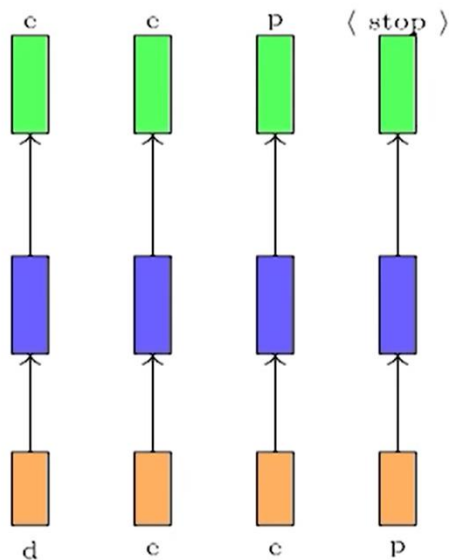
Objective of Lecture

- Explain the concept of recurrent computation and sequence modeling.
- Apply RNNs to real sequence problems.
- Analyze vanishing gradient issues.
- Implement

Sequence Learning Problem

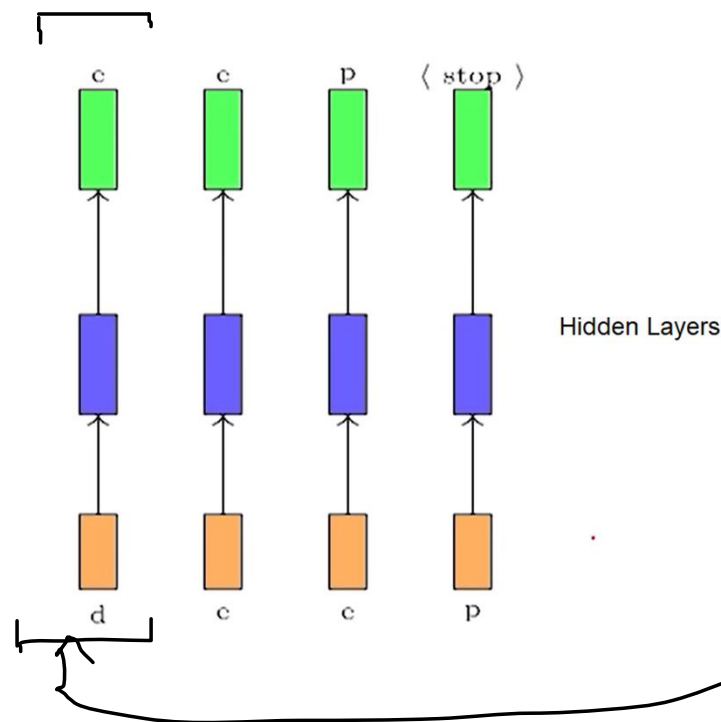
- Input: sequence $x_1, x_2, \dots, x_T$
- Output may be:
 - one value (e.g. classification)
 - a sequence (e.g. translation)

Sequence learning problem



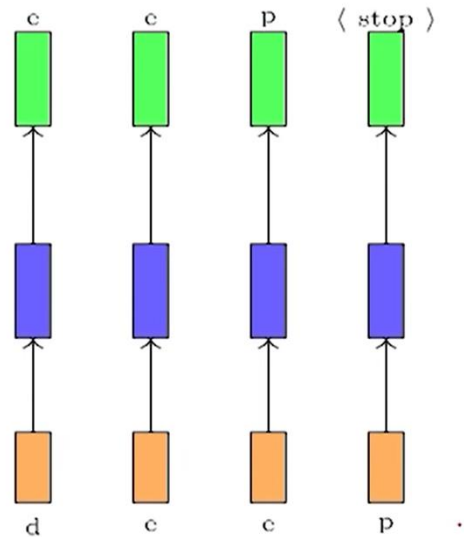
- In many applications the input is not of a fixed size
- Further successive inputs may not be independent of each other
- For example, consider the task of auto completion
- Given the first character 'd' you want to predict the next character 'e' and so on

Sequence learning problem (contd)



- Notice a few things
- First, successive inputs are no longer independent (while predicting 'e' you would want to know what the previous input was in addition to the current input)
- Second, the length of the inputs and the number of predictions you need to make is not fixed (for example, "learn", "deep", "machine" have different number of characters)
- Third, each network (orange-blue-green structure) is performing the same task (**input** : character **output** : character) (fully conn N/W)

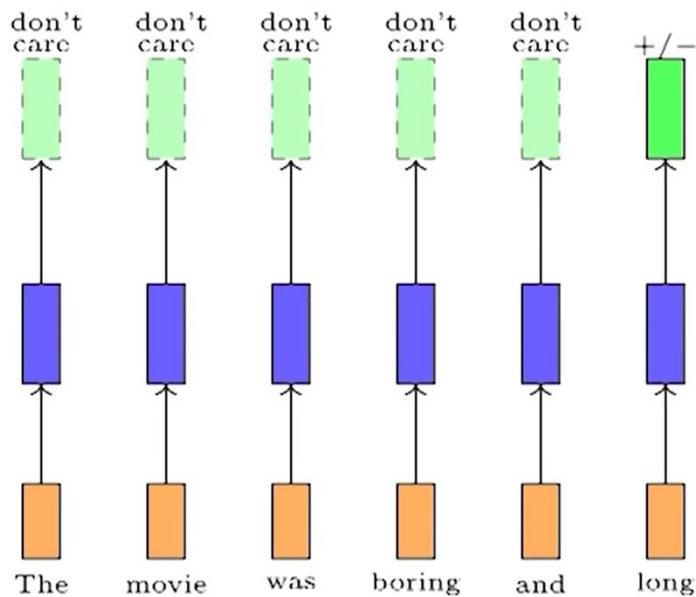
Sequence learning problem (contd)



t_1 t_2 t_3 t_4
each time step

- These are known as sequence learning problems
- We need to look at a sequence of (dependent) inputs and produce an output (or outputs)
- Each input corresponds to one time step
- Let us look at some more examples of such problems

Sequence learning problem (contd)



- Sometimes we may not be interested in producing an output at every stage
- Instead we would look at the full sequence and then produce an output
- For example, consider the task of predicting the polarity of a movie review
- The prediction clearly does not depend only on the last word but also on some words which appear before
- Here again we could think that the network is performing the same task at each step (input : word, output : +/-) but it's just that we don't care about intermediate outputs

RNNs for Sequence Labeling

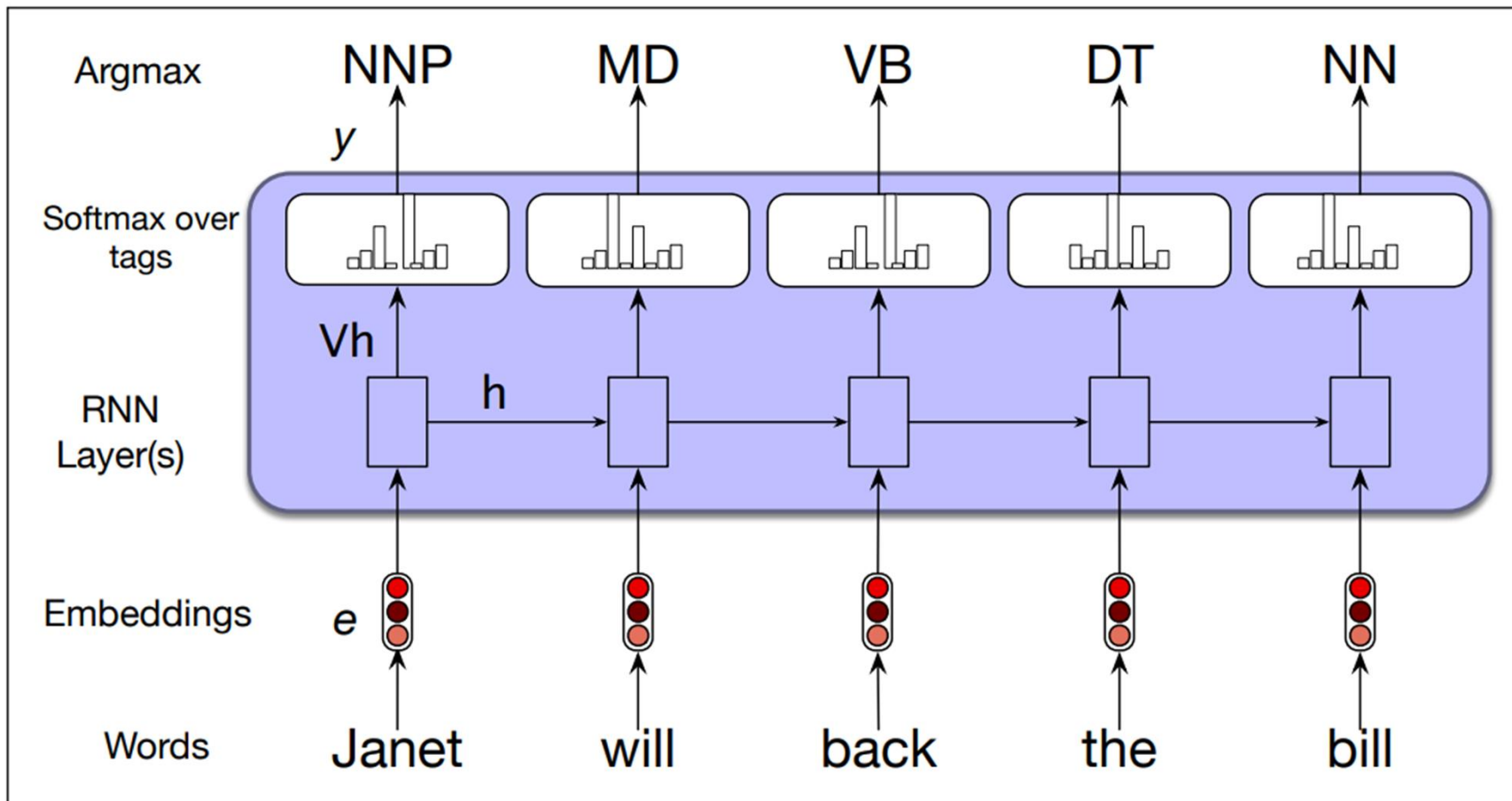


Figure 9.7 Part-of-speech tagging as sequence labeling with a simple RNN. Pre-trained word embeddings serve as inputs and a softmax layer provides a probability distribution over the part-of-speech tags as output at each time step.

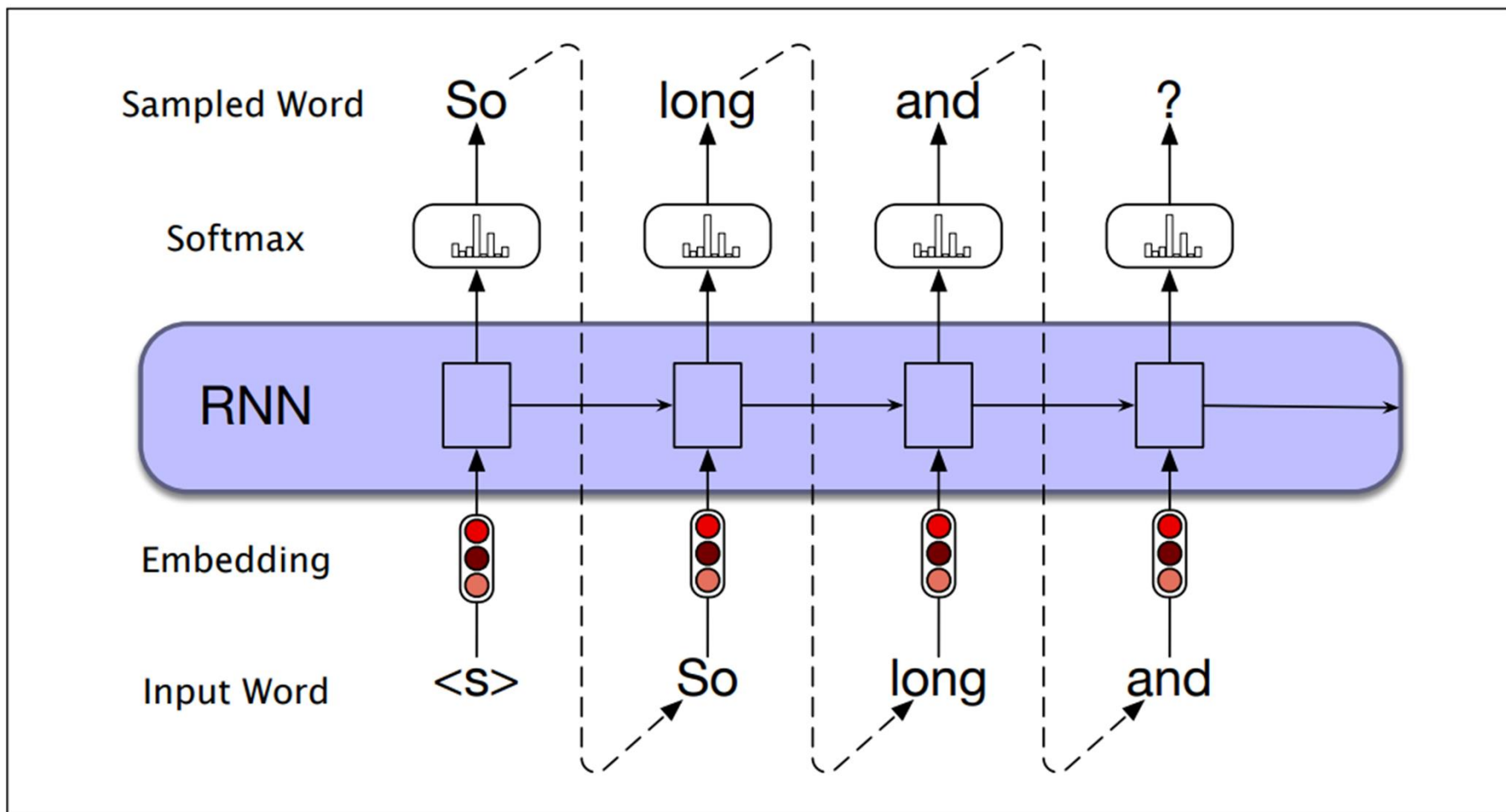
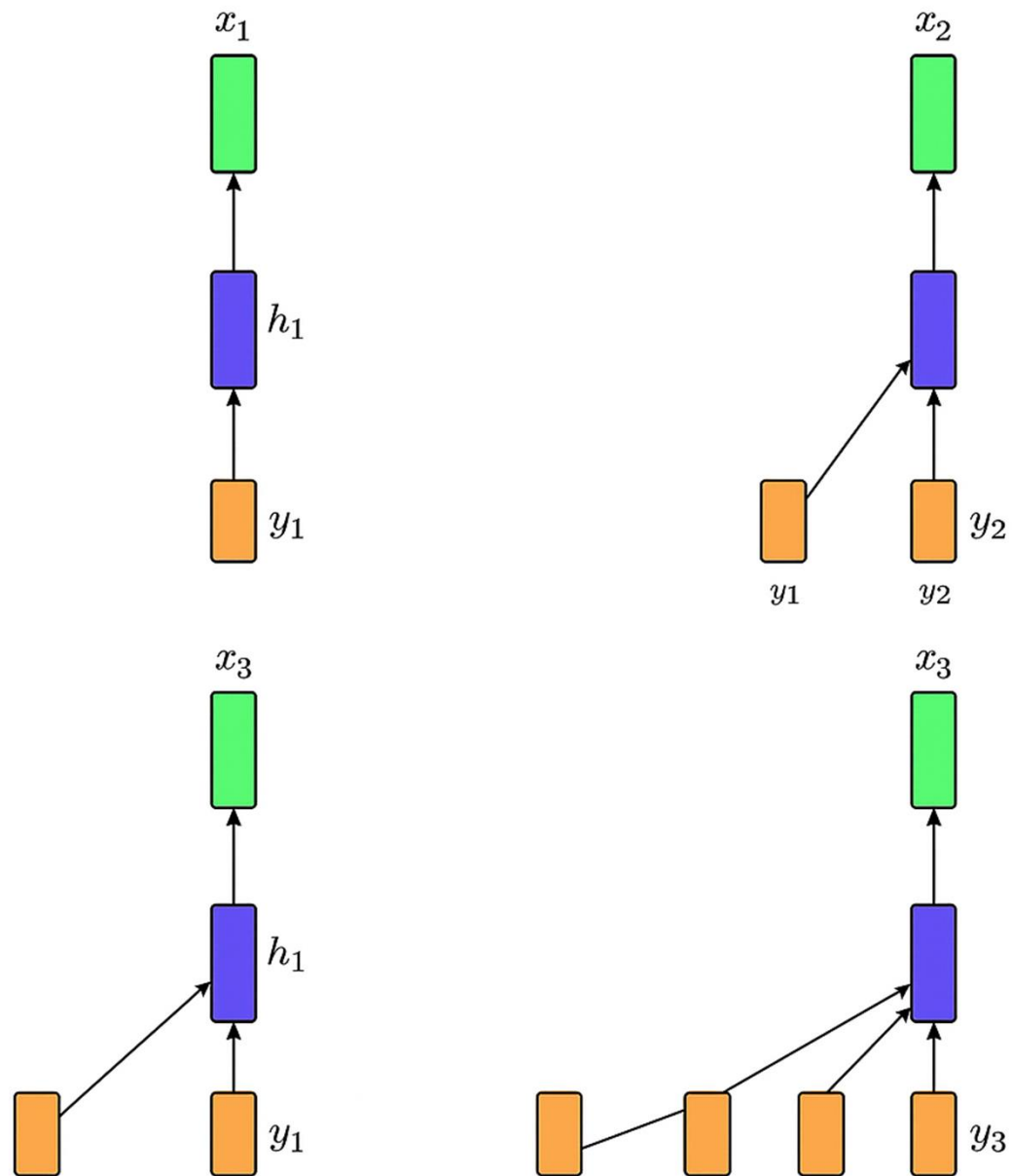


Figure 9.9 Autoregressive generation with an RNN-based neural language model.



- First, the function being computed at each time-step is now different:

$$y_1 = f_1(x_1)$$

$$y_2 = f_2(x_1, x_2)$$

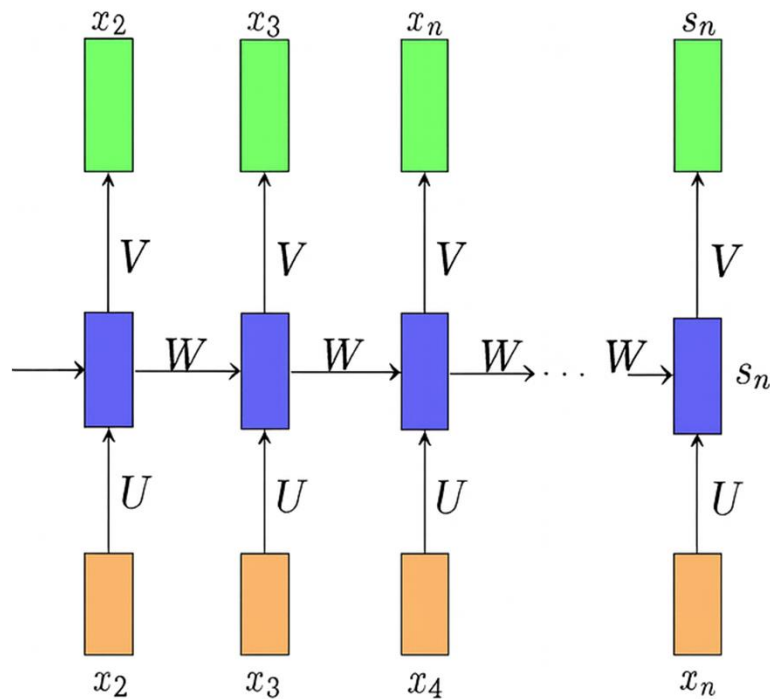
$$y_3 = f_3(x_1, x_2, x_3)$$

- The network is now **sensitive to the length of the sequence**.
- For example, a sequence of length 10 will require

$$f_1, \dots, f_{10}$$

whereas a sequence of length 100 will require

$$f_1, \dots, f_{100}$$



- The solution is to add a recurrent connection in the network,

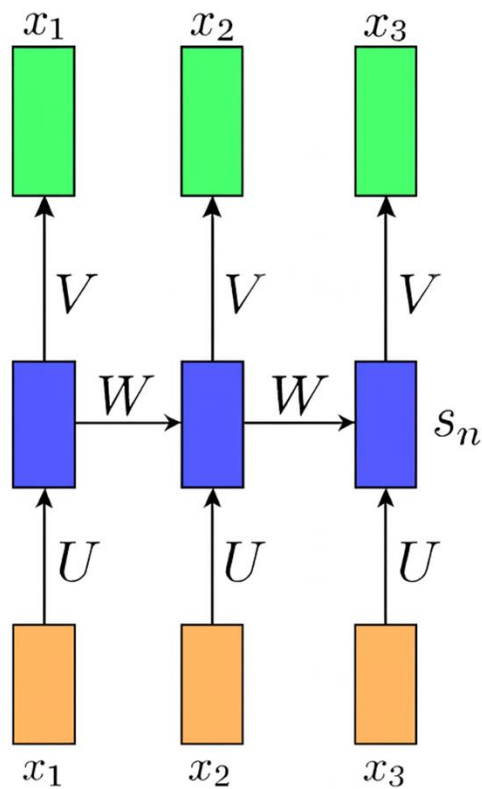
$$s_i = \sigma(Ux + Ws_{i-1} + b)$$

$$y_i = \sigma(Vs_i + c)$$

or

$$y_i = f(x_i, s_i, W, U, V)$$

- s_i is the state of the network at timestep i
- The parameters are W, U, V which are shared across timesteps
- The same network (and parameters) can be used to compute $y_1, y_2, \dots, y_n$ or y_{100}



$x_i \in \mathbb{R}^n$ (n-dimensional d)

$s_i \in \mathbb{R}^d$ (d-dimensional state)

$y_i \in \mathbb{R}^k$ (say k classes)

$U \in \mathbb{R}^{n \times d}$

$V \in \mathbb{R}^{d \times k}$

$W \in \mathbb{R}^{d \times d}$

Sequence Learning Illustration

Consider an RNN with the standard update

$$s_t = \tanh(Ux_t + Ws_{t-1} + b), \quad y_t = \sigma(Vs_t + c)$$

Use the following parameters and inputs:

- Input vectors (each 2×1):

$$x_1 = \begin{bmatrix} 0.5 \\ 1.0 \end{bmatrix}, \quad x_2 = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix}, \quad x_3 = \begin{bmatrix} 1.5 \\ 1.0 \end{bmatrix}$$

- Weight matrices & biases:

$$U = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \\ 0.5 & 0.6 \end{bmatrix}, \quad W = \frac{1}{2}I_3 = \begin{bmatrix} 0.5 & 0 & 0 \\ 0 & 0.5 & 0 \\ 0 & 0 & 0.5 \end{bmatrix}, \quad V = [0.7 \quad 0.8 \quad 0.9]$$

$$b = \begin{bmatrix} 0.1 \\ 0.1 \\ 0.1 \end{bmatrix}, \quad c = 0.1$$

- Activations:
 $\tanh(\cdot)$ for hidden, $\sigma(z) = 1/(1 + e^{-z})$ for output.
- Initial hidden state: $s_0 = \mathbf{0}$ (zero vector).

Task: compute s_1, s_2, s_3 and y_1, y_2, y_3 step-by-step.



1) Compute Ux_1

Rows of U times x_1 :

- row1: $0.1 \cdot 0.5 + 0.2 \cdot 1.0 = 0.05 + 0.20 = 0.25$
- row2: $0.3 \cdot 0.5 + 0.4 \cdot 1.0 = 0.15 + 0.40 = 0.55$
- row3: $0.5 \cdot 0.5 + 0.6 \cdot 1.0 = 0.25 + 0.60 = 0.85$

So

$$Ux_1 = \begin{bmatrix} 0.25 \\ 0.55 \\ 0.85 \end{bmatrix}.$$

Since $s_0 = \mathbf{0}$, $Ws_0 = \mathbf{0}$. Add bias b :

$$Ux_1 + Ws_0 + b = \begin{bmatrix} 0.25 \\ 0.55 \\ 0.85 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0.1 \\ 0.1 \end{bmatrix} = \begin{bmatrix} 0.35 \\ 0.65 \\ 0.95 \end{bmatrix}.$$

Apply $\tanh$ elementwise:

$$s_1 = \tanh \begin{bmatrix} 0.35 \\ 0.65 \\ 0.95 \end{bmatrix} \approx \begin{bmatrix} 0.33637554 \\ 0.57166997 \\ 0.73978305 \end{bmatrix}.$$

2) Compute y_1

First compute Vs_1 :

$$Vs_1 = 0.7 \cdot 0.33637554 + 0.8 \cdot 0.57166997 + 0.9 \cdot 0.73978305.$$

Calculate terms:

- $0.7 \times 0.33637554 = 0.23546288$
- $0.8 \times 0.57166997 = 0.45733598$
- $0.9 \times 0.73978305 = 0.66580475$

Sum: $0.23546288 + 0.45733598 + 0.66580475 = 1.35860361$.

Add bias $c = 0.1$: $z_1 = 1.35860361 + 0.1 = 1.45860361$.

Apply sigmoid:

$$y_1 = \sigma(z_1) = \frac{1}{1 + e^{-1.45860361}} \approx 0.81131901.$$

3) Compute s_2

Start with Ux_2 .

Rows of U times $x_2 = [1.0, 0.5]^T$:

- row1: $0.1 \cdot 1.0 + 0.2 \cdot 0.5 = 0.10 + 0.10 = 0.20$
- row2: $0.3 \cdot 1.0 + 0.4 \cdot 0.5 = 0.30 + 0.20 = 0.50$
- row3: $0.5 \cdot 1.0 + 0.6 \cdot 0.5 = 0.50 + 0.30 = 0.80$

So $Ux_2 = [0.2, 0.5, 0.8]^T$.

Compute recurrence term $Ws_1 = 0.5 \cdot s_1$ (since $W = 0.5I$):

$$Ws_1 = 0.5 \cdot \begin{bmatrix} 0.33637554 \\ 0.57166997 \\ 0.73978305 \end{bmatrix} = \begin{bmatrix} 0.16818777 \\ 0.28583499 \\ 0.36989153 \end{bmatrix}.$$

Now sum: $Ux_2 + Ws_1 + b$

$$= \begin{bmatrix} 0.2 \\ 0.5 \\ 0.8 \end{bmatrix} + \begin{bmatrix} 0.16818777 \\ 0.28583499 \\ 0.36989153 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0.1 \\ 0.1 \end{bmatrix} = \begin{bmatrix} 0.46818777 \\ 0.88583499 \\ 1.26989153 \end{bmatrix}.$$

Apply tanh:

$$s_2 = \tanh \begin{bmatrix} 0.46818777 \\ 0.88583499 \\ 1.26989153 \end{bmatrix} \approx \begin{bmatrix} 0.43673391 \\ 0.70933045 \\ 0.85376825 \end{bmatrix}.$$

Apply tanh:

$$s_2 = \tanh \begin{bmatrix} 0.46818777 \\ 0.88583499 \\ 1.26989153 \end{bmatrix} \approx \begin{bmatrix} 0.43673391 \\ 0.70933045 \\ 0.85376825 \end{bmatrix}.$$

4) Compute y_2

Compute $V s_2$:

$$V s_2 = 0.7 \cdot 0.43673391 + 0.8 \cdot 0.70933045 + 0.9 \cdot 0.85376825.$$

Terms:

- $0.7 \times 0.43673391 = 0.30571374$
- $0.8 \times 0.70933045 = 0.56746436$
- $0.9 \times 0.85376825 = 0.76839143$

Sum: $0.30571374 + 0.56746436 + 0.76839143 = 1.64156953$.

Add bias c : $z_2 = 1.64156953 + 0.1 = 1.74156953$.

Sigmoid:

$$y_2 = \sigma(z_2) \approx \frac{1}{1 + e^{-1.74156953}} \approx 0.85088631.$$

5) Compute s_3

Compute Ux_3 for $x_3 = [1.5, 1.0]^T$:

- row1: $0.1 \cdot 1.5 + 0.2 \cdot 1.0 = 0.15 + 0.20 = 0.35$
- row2: $0.3 \cdot 1.5 + 0.4 \cdot 1.0 = 0.45 + 0.40 = 0.85$
- row3: $0.5 \cdot 1.5 + 0.6 \cdot 1.0 = 0.75 + 0.60 = 1.35$

So $Ux_3 = [0.35, 0.85, 1.35]^T$.

Recurrence: $Ws_2 = 0.5 \cdot s_2 \approx [0.21836696, 0.35466522, 0.42688412]^T$.

Sum with bias:

$$Ux_3 + Ws_2 + b = \begin{bmatrix} 0.35 \\ 0.85 \\ 1.35 \end{bmatrix} + \begin{bmatrix} 0.21836696 \\ 0.35466522 \\ 0.42688412 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0.1 \\ 0.1 \end{bmatrix} = \begin{bmatrix} 0.66836696 \\ 1.30466522 \\ 1.87688412 \end{bmatrix}.$$

Apply tanh:

$$s_3 = \tanh \begin{bmatrix} 0.66836696 \\ 1.30466522 \\ 1.87688412 \end{bmatrix} \approx \begin{bmatrix} 0.58390464 \\ 0.86291933 \\ 0.95421415 \end{bmatrix}.$$

6) Compute y_3

Compute V_{s_3} :

$$V_{s_3} = 0.7 \cdot 0.58390464 + 0.8 \cdot 0.86291933 + 0.9 \cdot 0.95421415.$$

Terms:

- $0.7 \times 0.58390464 = 0.40873325$
- $0.8 \times 0.86291933 = 0.69033546$
- $0.9 \times 0.95421415 = 0.85879273$

$$\text{Sum: } 0.40873325 + 0.69033546 + 0.85879273 = 1.95786144.$$

$$\text{Add bias } c: z_3 = 1.95786144 + 0.1 = 2.05786144.$$

Sigmoid:

$$y_3 = \sigma(z_3) \approx \frac{1}{1 + e^{-2.05786144}} \approx 0.88673957.$$